Michael Galileo Hasian (201344020)	**Rangkuman Minggu ke-2**	Sabtu, 3 September 2022
		Aplikasi Mikrokontroler
3-TNK		Ferry Satria, BSEE, M.T. Rahmawati Hasanah, S.ST.,M.T.

1. Status Register 8535

Status Register merupakan Register spesial yang berfungsi untuk menampung nilai ‘1’ dan ‘0’, yang mana nilainya didapatkan ketika program menerima fungsi tertentu atau hasil. Berikut jenis-jenis dari SREG pada ATMega 8535 :

- I = Interrupt. Saat I = 1 (Enable), dan jika I = 0 (Disable)
- T = Menyimpan data 1 bit saat menggunakan BST (Bit Store), dan jika ingin mengambil data menggunakan BLD (Bit Load).
- H = Half Carry. Carry yang terjadi dari posisi nibble (4 bit) LOW (LN) atau HIGHnya (HN). Untuk menyimpan carry yang berada ditengah-tengah byte (8 bit).
- S = Sign Flag. Untuk membackup Sign-Bit pada Negative Flag bila terjadi overflow yang menyebabkan data error. Cara kerjanya ialah dengan melakukan operasi NOR sebagai berikut: $S = N \oplus V$ Saat S = 1 (Negative) ; S = 0 (Positive)
- V = Overflow. Menampung nilai hasil operasi yang melebihi batas ukuran pada suatu register. Semisal register dapat menampung 8 bit, lalu saat proses penjumlahan ternyata membuahkan hasil 9 bit, 1 bit kelebihannya masuk kedalam overflow. Dalam oeprasi matematis overflow biasanya terjadi ketika nilai nya memiliki
- N = Negative Flag. Berfungsi seperti Sign Flag, namun kebenarannya dapat dipengaruhi oleh overflow sehingga hasilnya dapat berupa kesalahan.
- Z = Zero Flag.
- C = Carry Flag.

2. Penjumlahan Desimal pada Mikrokontroler ATMega 8535

Mikrokontroler ATMega 8535 sama seperti Mikroprosesor 8086 yaitu bekerja dengan sistem bilangan biner, sehingga untuk melakukan penjumlahan bilangan desimal diperlukan proses konversi bilangan dari heksadesimal menjadi desimal. Berbeda dengan Mikroprosesor 8086 yang memiliki perintah DAA (*Decimal Adjust after Addition*),

Mikrokontroler ATMega 8535 tidak memiliki instruksi khusus seperti DAA sehingga perlu membuat sebuah subrutin DAA sendiri, berikut subrutin DAA yang dapat digunakan di Mikrokontroler ATMega 8535 :

```
;subrutin unt koreksi hasil penjumlahan mjd. BCD.2 dgt.
; yang dilaksanakan mll. r19 sbg. acc.:
daa:    push r16;amankan isi reg. ke stack
        push r17
        push r21
        push r24
        in r16,sreg; amankan sreg
        in r17,sreg
        ldi r24,0 ; r24 hanya sbg status dr carry flag
        adc r24,r24; status carry flag sebelum koreksi
; ambil, amankan kembali sreg, uji half carry
        out sreg,r17
        brhc lup4; lompat ke lup4 jika H=0
        rcall fk_ln; satuan + 06 jika H=1
    ;   rjmp lup5
lup4:   mov r21,r19
        andi r21,$0f; ambil dgt.satuan
        cpi r21,$0a ; > 9
        brcs lup5; lompat ke lup5 jika < 9
        rcall fk_ln; satuan + 06 jika > 9
lup5:   out sreg,r17; ambil, amankan kembali sreg, uji carry
        brcc lup6; lompat ke lup6 jika C=0
        rcall fk_hn; puluhan + 60 jika C=1
    ;   rjmp lup7
lup6:   mov r21,r19
        andi r21,0xf0; ambil dgt.puluhan
        cpi r21,0xa0
        brcs lup7; lompat ke lup7 jika pul < a
        rcall fk_hn; pul + 60 jika > 9
lup7:   out sreg,r16; mengatur carry flag sblm keluar:
```

```
        clc; C = 0
        in r16,sreg ;amankan sreg stlh c=0
        cpi r24,0 ;menguji status
        breq lup8
        out sreg,r16;ambil sreg, set c
        sec; C = 1
        in r16,sreg; amankan sreg stlh c=1
lup8:   out sreg,r16;status akhir
        pop r24;kembalikan isi reg. dr stack
        pop r21
        pop r17
        pop r16
        ret
fk_ln:  ldi r21,06; koreksi dgt.satuan
        out sreg,r16
        add r19,r21
        in r16,sreg
        brcc tuh
        ldi r21,0x01
        add r17,r21
tuh:    out sreg,r16
        adc r24,r24
        ret
fk_hn:  ldi r21,$60; koreksi dgt.puluhan
        out sreg,r16
        add r19,r21
        in r16,sreg
        adc r24,r24; amankan carry pd r24
        ret ; akhir dari prog. subrutin.
```

Catatan :

- Subrutin DAA ini dapat dipanggil menggunakan perintah “rcall” (relative call).
- Subrutin DAA ini melibatkan r19, sehingga data hasil penjumlahan yang akan dikoreksi harus dipindahkan dahulu ke r19.

- Subrutin DAA hanya dapat dieksekusi jika ada perintah penjumlahan “ADD” atau “ADC” sebelumnya.

3. Program Penjumlahan Desimal

Buat program untuk menjumlahkan 2 bilangan desimal berikut :

a. 78 + 95, data input disimpan pada register r3 : r2

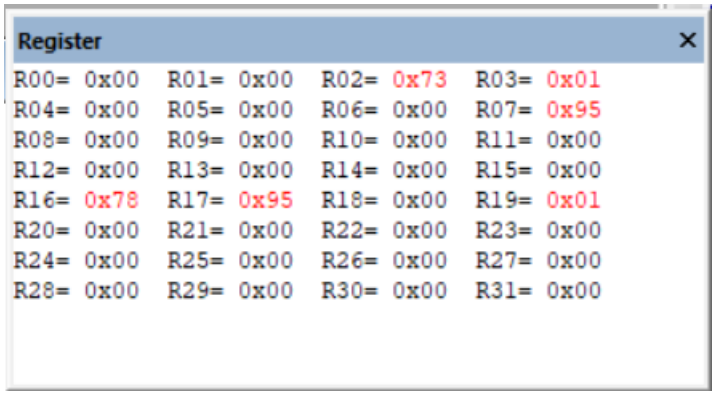
- **Kode Program**

```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r16,low(ramend)
out spl,r16             ; spl = stack pointer low
ldi r16,high(ramend)
out sph,r16

ldi r16,0x78
ldi r17,0x95
mov r2,r16
mov r7,r17
add r2,r7
mov r19,r2
rcall daa
mov r2,r19
mov r19,r3
adc r19,r1
mov r3,r19

akhir: rjmp akhir
;*subrutin DAA diinputkan di sini
```

- **Output Program**

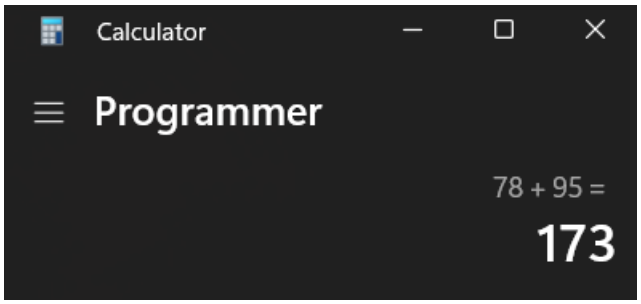


Register Input Data = Data Pertama = R0 = 78

Data Kedua = R1 = 95

Register Output Data = R3 : R2 = 01 : 73

- **Pembuktian**



b. 8937 + 7899, data input disimpan pada register r7 : r6 : r5

- **Kode Program**

```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r16,low(ramend)
out spl,r16             ; spl = stack pointer low
ldi r16,high(ramend)
out sph,r16

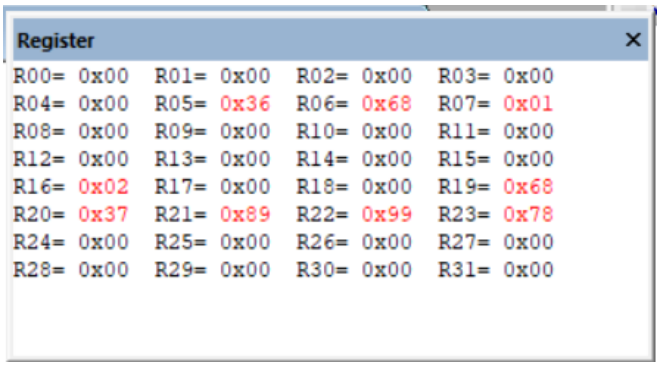
ldi r20,0x37
```

```
ldi r21,0x89  ; data pertama = 8937
ldi r22,0x99
ldi r23,0x78  ; data kedua = 7899

mov r19,r20
add r19,r22
rcall daa
mov r5,r19
mov r19,r21
adc r19,r23
rcall daa
mov r6,r19
adc r7,r1

akhir: rjmp akhir
;*subrutin DAA diinputkan di sini
```

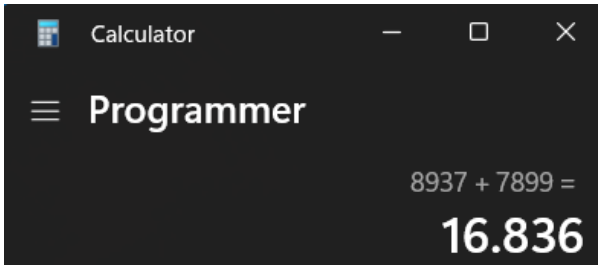
- **Output Program**



Register Input Data : Data Pertama = R21 : R20 = 89 : 37
Data Kedua = R23 : R22 = 78 : 99

Register Output Data : R7 : R6 : R5 = 01 : 68 : 36

- **Pembuktian**



c. 78 + 95, data input disimpan pada data area

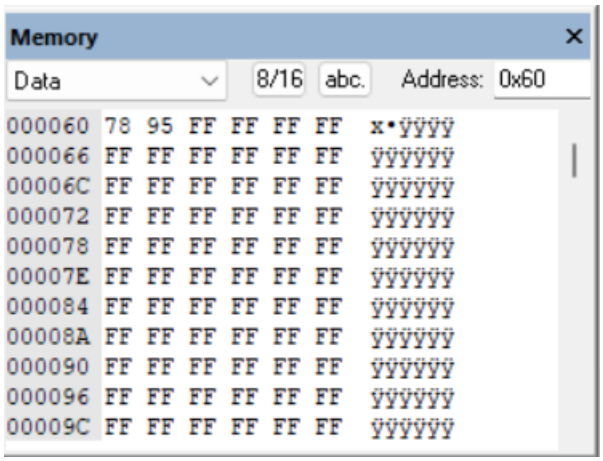
- **Kode Program**

```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r16,low(ramend)
out spl,r16             ; spl = stack pointer low
ldi r16,high(ramend)
out sph,r16

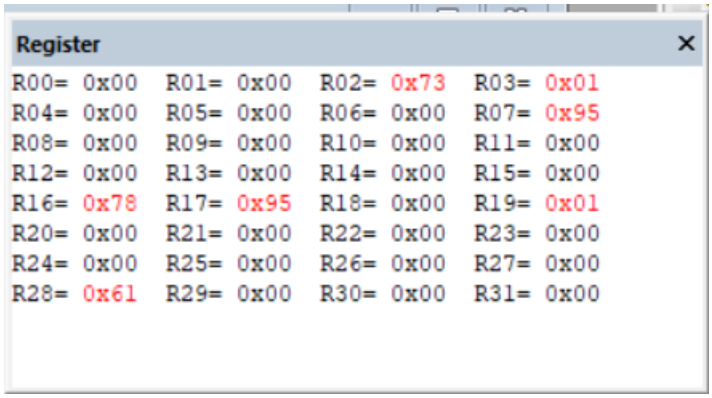
ldi r28,0x60
ld r16,y+
ld r17,y
mov r2,r16
mov r7,r17
add r2,r7
mov r19,r2
rcall daa
mov r2,r19
mov r19,r3
adc r19,r1
mov r3,r19

akhir: rjmp akhir
;*subrutin DAA diinputkan di sini
```

- **Output Program**



Data Input = 78 dan 95



Register Data Ouput = R3 : R2 = 01 : 73

d. 78 + 95, data input disimpan pada program area

- **Kode Program**

```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r16,low(ramend)
out spl,r16             ; spl = stack pointer low
ldi r16,high(ramend)
out sph,r16
```

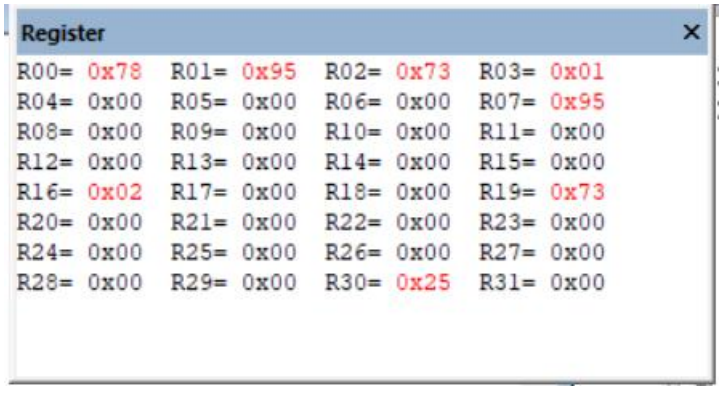
```
ldi zl,low(2*dat)
ldi zh,high(2*dat)
lpm r0,Z
inc r30
lpm r1,Z

mov r2,r0
mov r7,r1
add r2,r7
mov r19,r2
rcall daa
mov r2,r19
adc r3,r28

akhir: rjmp akhir

dat:
.db 0x78, 0x95
;*subrutin DAA diinputkan di sini
```

- **Output Program**



Register Input Data = Data Pertama = R0 = 78

Data Kedua = R1 = 95

Register Output Data = R3 : R2 = 01 : 73

e. 8937 + 7899, data input disimpan pada data area

- **Kode Program**

```
..include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r16,low(ramend)
out spl,r16             ; spl = stack pointer low
ldi r16,high(ramend)
out sph,r16

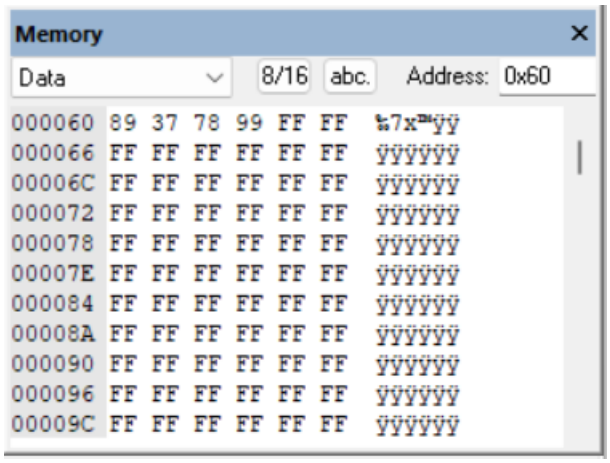
ldi r28,0x60
ld r21,y+
ld r20,y+
ld r23,y+
ld r22,y

mov r19,r20
add r19,r22
rcall daa
mov r5,r19

mov r19,r21
adc r19,r23
rcall daa
mov r6,r19

adc r7,r1
akhir: rjmp akhir;*subrutin DAA diinputkan di sini
```

- **Output Program**



Input Data Pertama = 89 : 37

Input Data Kedua = 78 : 99



Register Output Data : R7 : R6 : R5 = 01 : 68 : 36

f. 8937 + 7899, data input disimpan pada program area

- **Kode Program**

```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r16,low(ramend)
out spl,r16             ; spl = stack pointer low
ldi r16,high(ramend)
out sph,r16
```

```
ldi zl,low(2*dat)
ldi zh,high(2*dat)
lpm r20,Z+
lpm r21,Z+
lpm r22,z+
lpm r23,z

mov r19,r20
add r19,r22
rcall daa
mov r5,r19

mov r19,r21
adc r19,r23
rcall daa
mov r6,r19
adc r7,r1

akhir: rjmp akhir
dat:
.db 0x37,0x89,0x99,0x78
;*subrutin DAA diinputkan di sini
```

- **Output Program**



Register Output Data : R7 : R6 : R5 = 01 : 68 : 36

4. Program Konversi Bilangan Heksadesimal ↔ Desimal

1. Konversikan bilangan heksadesimal 2 digit yang terdapat pada register r20 menjadi desimal. Tempatkan hasil konversi pada register r22:r21.

- **Kode Program**

```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r20,low(ramend)
out spl,r20
ldi r20,high(ramend)
out sph,r20

ldi r16,4                       ; counter untuk penggandaan
ldi r20,0xff                    ; register input data
mov r25,r20
andi r25,0xf0                   ; mengambil nilai 16-an
lsr r25
lsr r25
lsr r25
lsr r25
mov r19,r25
add r19,r0
rcall daa
mov r21,r19
lup:mov r19,r21                 ; lup untuk menggandakan
        add r19,r19
        rcall daa
        mov r21,r19
        mov r19,r22
        adc r19,r19
        rcall daa
        mov r22,r19
```

```
        dec r16
        brne lup
mov r25,r20
andi r25,0x0f                   ; mengambil nilai satuan
mov r19,r25
add r19,r0
rcall daa
add r19,r21
rcall daa
mov r21,r19
mov r19,r22
adc r19,r0
rcall daa
mov r22,r19

akhir: rjmp akhir

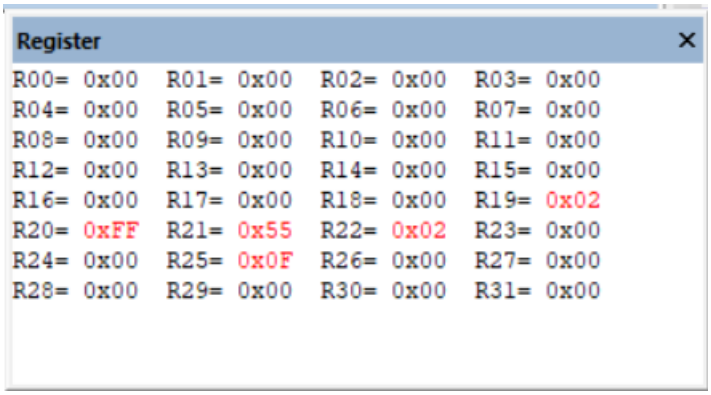


; subrutin unt koreksi hasil penjumlahan mjd. BCD.2 dgt.
; yang dilaksanakan mll. r19 sbg. acc.:
daa:  push r16;amankan isi reg. ke stack
                push r17
                push r21
                push r24
                in r16,sreg; amankan sreg
                in r17,sreg
                ldi r24,0 ; r24 hanya sbg status dr carry flag
                adc r24,r24; status carry flag sebelum koreksi
; ambil, amankan kembali sreg, uji half carry
                out sreg,r17
                brhc lup4; lompat ke lup4 jika H=0
                rcall fk_ln; satuan + 06 jika H=1
        ;       rjmp lup5
lup4: mov r21,r19
```

```
                andi r21,$0f; ambil dgt.satuan
                cpi r21,$0a ; > 9
                brcs lup5; lompat ke lup5 jika < 9
                rcall fk_ln; satuan + 06 jika > 9
lup5: out sreg,r17; ambil, amankan kembali sreg, uji carry
                brcc lup6; lompat ke lup6 jika C=0
                rcall fk_hn; puluhan + 60 jika C=1
        ;       rjmp lup7
lup6: mov r21,r19
                andi r21,0xf0; ambil dgt.puluhan
                cpi r21,0xa0
                brcs lup7; lompat ke lup7 jika pul < a
                rcall fk_hn; pul + 60 jika > 9
lup7: out sreg,r16; mengatur carry flag sblm keluar:
                clc; C = 0
                in r16,sreg ;amankan sreg stlh c=0
                cpi r24,0 ;menguji status
                breq lup8
                out sreg,r16;ambil sreg, set c
                sec; C = 1
                in r16,sreg; amankan sreg stlh c=1
lup8:
                out sreg,r16;status akhir
                pop r24;kembalikan isi reg. dr stack
                pop r21
                pop r17
                pop r16
                ret
fk_ln:          ldi r21,06; koreksi dgt.satuan
                out sreg,r16
                add r19,r21
                in r16,sreg
                brcc tuh
                ldi r21,0x01
```

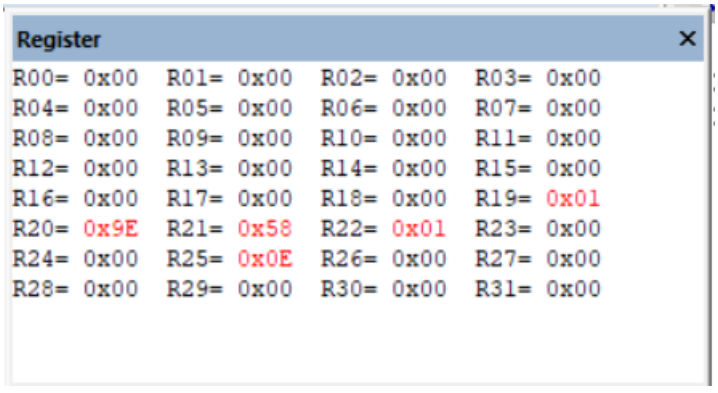
```
                add r17,r21
tuh:   out sreg,r16
                adc r24,r24
                ret
fk_hn:          ldi r21,$60; koreksi dgt.puluhan
                out sreg,r16
                add r19,r21
                in r16,sreg
                adc r24,r24; amankan carry pd r24
                ret ; akhir dari prog. subrutin.
```

- **Output Program**



Register Input Data = R20 = FF

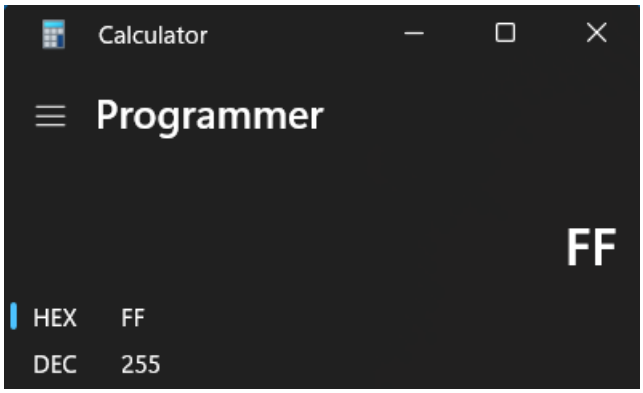
Register Output Data = R22 : R21 = 2 : 55

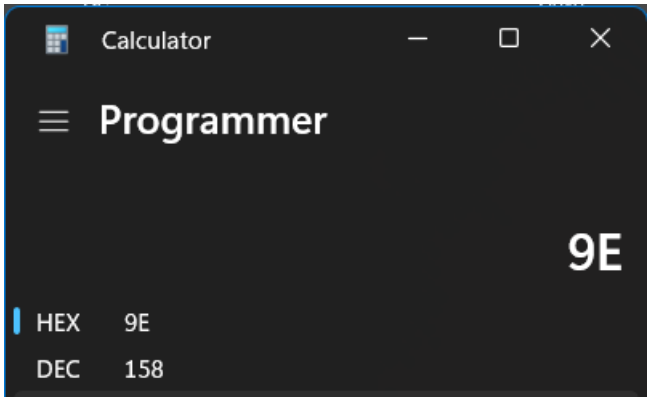


Register Input Data = R20 = 9E

Register Output Data = R22 : R21 = 1 : 58

- **Pembuktian**





2. Konversikan bilangan desimal 2 digit yang berada pada register r16 menjadi heksadesimal. Simpan hasil konversi pada register r17.

- **Kode Program**

```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r20,low(ramend)
out spl,r20
ldi r20,high(ramend)
out sph,r20

ldi r16,0x77            ; register input data
mov r20,r16
andi r20,0xf0                   ; mengambil nilai puluhan
lsr r20
```

```
lsr r20
lsr r20
lsr r20
ldi r21,0x0a
mul r20,r21
mov r17,r0

mov r20,r16
andi r20,0x0f

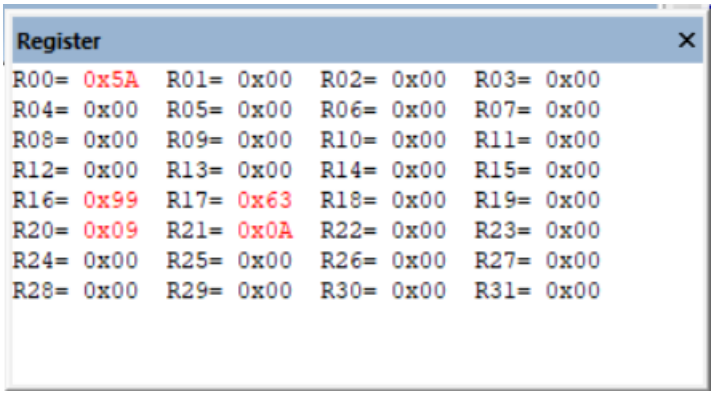
add r17,r20
akhir: rjmp akhir
```

- **Output Program**



Register Input Data = R16 = 53

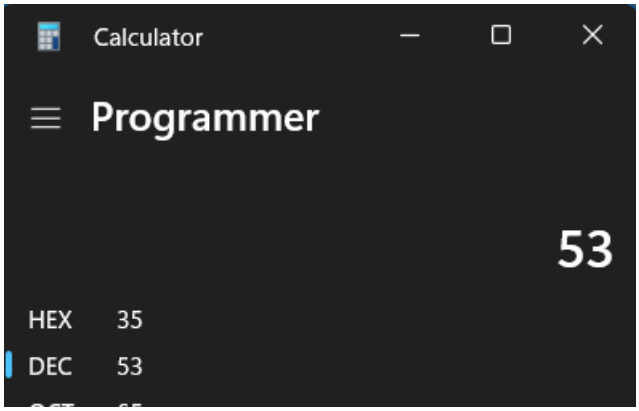
Register Output Data = R17 = 35

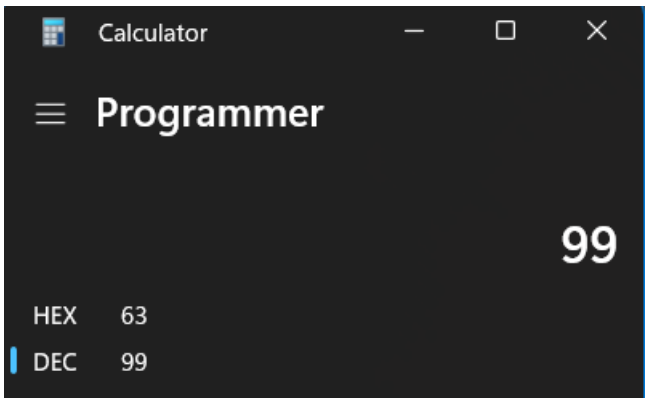


Register Input Data = R16 = 99

Register Output Data = R17 = 63

- **Pembuktian**





3. Konversikan bilangan desimal 4 digit yang berada pada data area menjadi heksadesimal. Simpan hasil konversi pada register r22:r21.

- **Kode Program**

```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r20,low(ramend)
out spl,r20
ldi r20,high(ramend)
out sph,r20

ldi r28,0x60            ; input data dari data area
ld r17,y+
```

```
ld r16,y

mov r18,r17
mov r19,r16
andi r18,0xf0          ; mengambil nilai ribuan (d1)
lsr r18
lsr r18
lsr r18
lsr r18
ldi r31,0x0a       ; pengali
mul r18,r31            ; d1*0a
mov r21,r0
mov r22,r1
mov r18,r17
andi r18,0x0f          ; mengambil nilai ratusan (d2)
add r21,r18            ; d1*0a + d2
adc r22,r30

mul r21,r31            ; (((d1*0a)+d2)*0a)
mov r21,r0
mov r22,r1             ; (((d1*0a)+d2)*0a)
andi r19,0xf0          ; mengambil nilai d3
lsr r19
lsr r19
lsr r19
lsr r19
add r21,r19            ; (((d1*0a)+d2)*0a)+d3
adc r22,r30

mul r21,r31
mov r2,r0
mov r3,r1

mul r22,r31
```

```
rcall gnd
add r0,r2
adc r1,r3                ; ((((d1*0a)+d2)*0a)+d3)*0a
mov r21,r0
mov r22,r1               ; ((((d1*0a)+d2)*0a)+d3)*0a

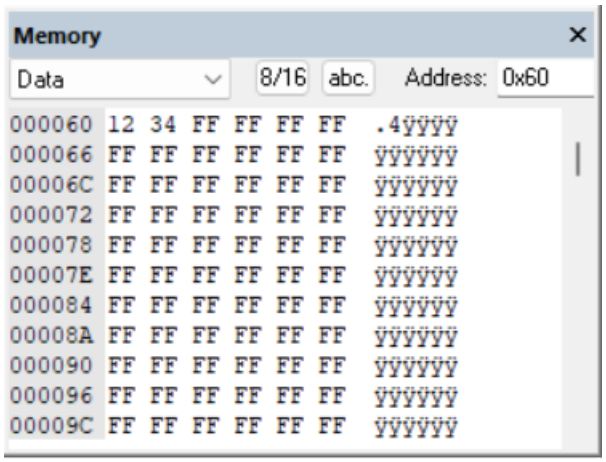
mov r19,r16
andi r19,0x0f            ; mengambil nilai d4

add r21,r19
adc r22,r30              ; (((((d1*0a)+d2)*0a)+d3)*0a)+d4

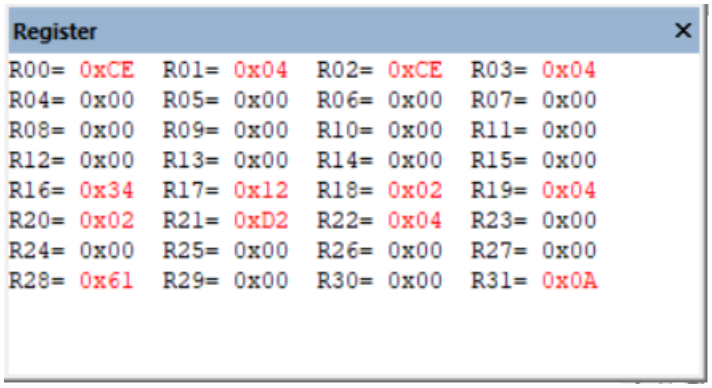
akhir: rjmp akhir

gnd:                     ; subrutin untuk menggeser sebanyak 8 kali ke kiri
ldi r25,8
l1:   add r0,r0          ; register r0 dan r1
      adc r1,r1
      dec r25
      brne l1
ret
```

- **Output Program**

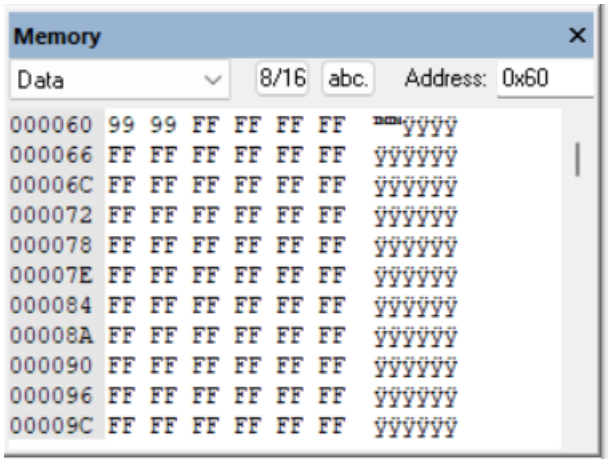


Data Input = 1234



Register Data Input = R17 : R16 = 12 : 34

Register Data Output = R22 : R21 = 4 : D2



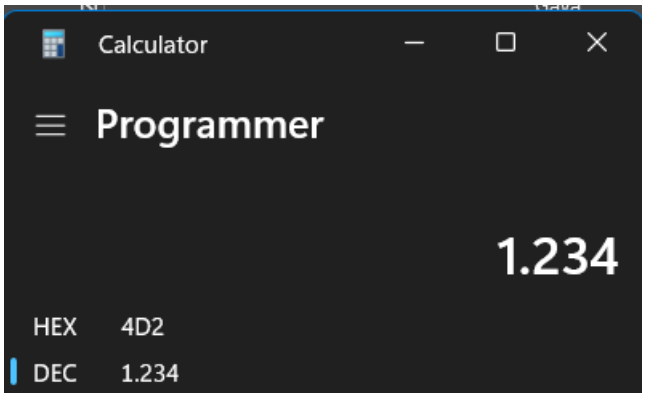
Data Input = 9999

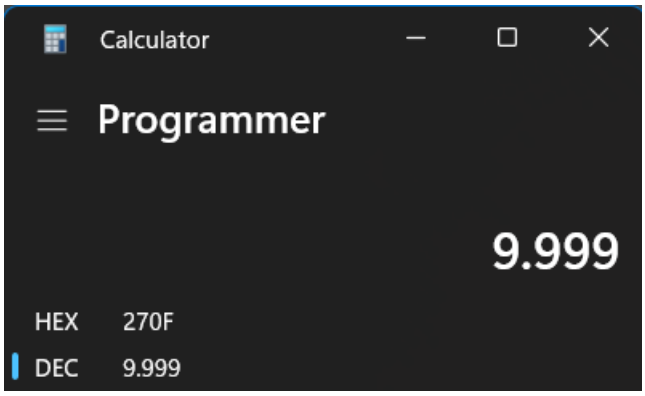


Register Data Input = R17 : R16 = 99 : 99

Register Data Output = R22 : R21 = 27 : 0F

- **Pembuktian**


Calculator
Programmer
1.234
HEX 4D2
DEC 1.234


Calculator
Programmer
9.999
HEX 270F
DEC 9.999